

Answer key — finished scripts and challenge solutions.

Welcome to Scratch G - Space Invaders. This is the capstone: a full arcade game, shown here as a *finished reference build* you can assemble and then make your own. It pulls together everything - movement, variables, conditionals, custom blocks, and `broadcast` - and adds the one idea that makes big Scratch projects possible: **clone coordination**. A single **Invader** sprite stamps out 55 clones, and a few shared variables plus a broadcast *beat* make the whole fleet march, drop, and turn as one. The rule to say out loud: *if there are lots of them on screen, they are clones*. Open scratch.mit.edu, click *Create*, and build the invasion. The costumes are in `scratch-invaders/assets/sliced/` (run `slice_sprite.py`).

The block groups you will use. Each block has a colour. The colour tells you which group it is in. Find the colour first, then find the block.

Motion	(blue)	Move the sprite: move, turn, go to, point towards, change x or y.
Looks	(purple)	Change how the sprite looks: say, costume, size, backdrop.
Sound	(pink)	Play a sound.
Events	(yellow)	Start a script (green flag, key, click, when I receive) and broadcast messages.
Control	(orange)	Choose when and how often blocks run: wait, forever, repeat, if, stop.
Sensing	(light blue)	Ask about the world: touching?, key pressed?, mouse, ask and answer.
Operators	(green)	Do maths and comparisons: plus, pick random, less-than, equals, more-than, and/or.
Variables	(dark orange)	Remember a value like score: set, change, and show a variable.

Project 1: The ship

Drive the player ship left and right along the bottom row, and keep it on the screen. Unlike the Dino, the ship DOES move sideways - but the fleet will still come down to meet it.

Get ready. Add a **Player** sprite and upload `ship_fighter_1` as its costume. Make a variable **score** *For all sprites*. The ship lives on one row near the bottom (`y = -150`).

New blocks.

<code>if <key () pressed?></code>	Reading the keys inside one forever loop moves the ship smoothly, in either direction.
<code>set x to ()</code>	A guard: clamp <code>x position</code> back to the edge so the ship can't slide off the stage.

Build it.

1. On the **Player**: `when green flag clicked`, `show`, `go to x: (0) y: (-150)`, then `set [score] to (0)`.

2. Add a **forever** loop. Inside it, two key checks: **if <key [left arrow] pressed?> then change x by (-6)**, and the same for **right arrow** with **change x by (6)**.
3. Still in the loop, two guards: **if <(x position) > (220)> then set x to (220)**, and **if <(x position) < (-220)> then set x to (-220)**.
4. Green flag, and drive with the arrow keys.

When you are done, your code should look like this:



Player: drive left/right, stay on screen

Reading keys with **if <key () pressed?>** inside a forever loop (not separate **when key pressed** hats) is what makes the ship feel responsive - the same trick Maze Quest used.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Tune the feel: if the ship is too slow, raise both moves to (8); too twitchy, lower them to (4).

Expected: The ship's speed matches your taste.

2. Swap the look: try `ship_delta_1` as the costume instead.

Expected: The classic arrow-fighter shape.

Project 2: Shooting

Fire a laser upward with the space bar - but only **ONE** at a time, just like the 1978 arcade. The laser is a clone; a state flag `bulletActive` stops you spraying the screen.

Get ready. Add a **Bullet** sprite (costume `bullet_cyan_1`). Make a variable `bulletActive` For all sprites. Start the sprite hidden - only its clones ever show.

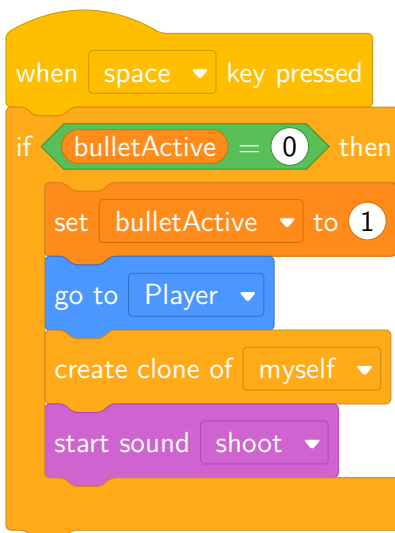
New blocks.

<code>go to [Player]</code>	Jump the bullet to the ship's muzzle before cloning, so the shot starts where the ship is.
<code>create clone of [myself]</code>	Spawn a copy that runs the <code>when I start as a clone</code> script - your flying laser.
<code>delete this clone</code>	A clone's own life: it runs the clone hat, then cleans itself up when the shot is spent.

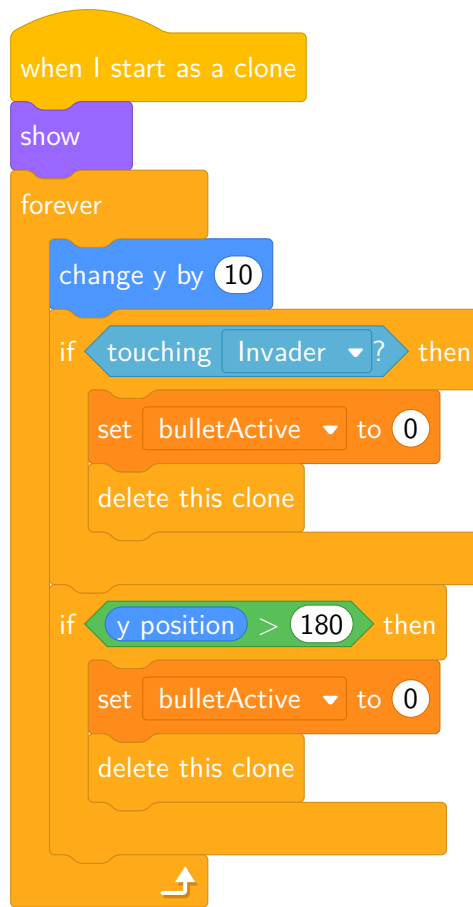
Build it.

1. On the **Bullet**: `when green flag clicked`, `hide`, `set [bulletActive] to (0)`.
2. Add `when [space] key pressed`. Inside `if <(bulletActive) = (0)>`: `set [bulletActive] to (1)`, `go to [Player]`, `create clone of [myself]`, `start sound [shoot]`.
3. Add `when I start as a clone`: `show`, then a `forever` that does `change y by (10)`; if `<touching [Invader]?>` `set bulletActive back to (0)` and `delete this clone`; and if the laser flies off the top `<(y position) > (180)>`, do the same.
4. Green flag: fire with space. You get one shot at a time - the next only after this one lands or leaves the top.

The one-shot fire rule (left) and each laser's flight (right):



Bullet: fire, but only if none is flying



Bullet: fly up, then clean up

`bulletActive` is the same state-flag idea as the Dino's `grounded?`: a 0/1 variable that gates an action so it can only happen when it should.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Machine-gun mode: temporarily remove the `if <(bulletActive) = (0)>` wrapper and hold space.

Expected: A wall of lasers - now you see why the flag matters.

2. Colour your shot: swap the costume to `bullet_orange_1` or `bullet_green_1`.

Expected: The laser matches your theme.

Project 3: Build the fleet

Stamp out the classic $5 \times 11 = 55$ invaders from ONE sprite, using a custom block `spawn invader (x) (y)` called inside two nested loops. This is My Blocks with inputs, put to real work.

Get ready. Add an **Invader** sprite with two wiggle costumes, e.g. `invader_green_1` and `invader_green_2`. Make variables `invadersLeft`, `invaderDir`, `invaderStep`, `row`, `col` (all *For all sprites*) and, *For this sprite only*, `me`.

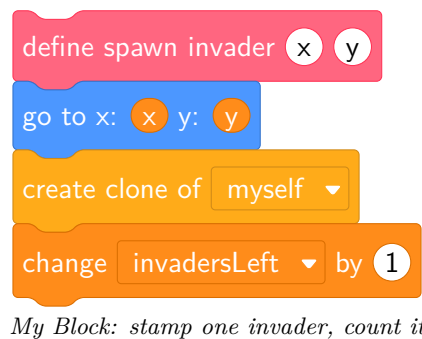
New blocks.

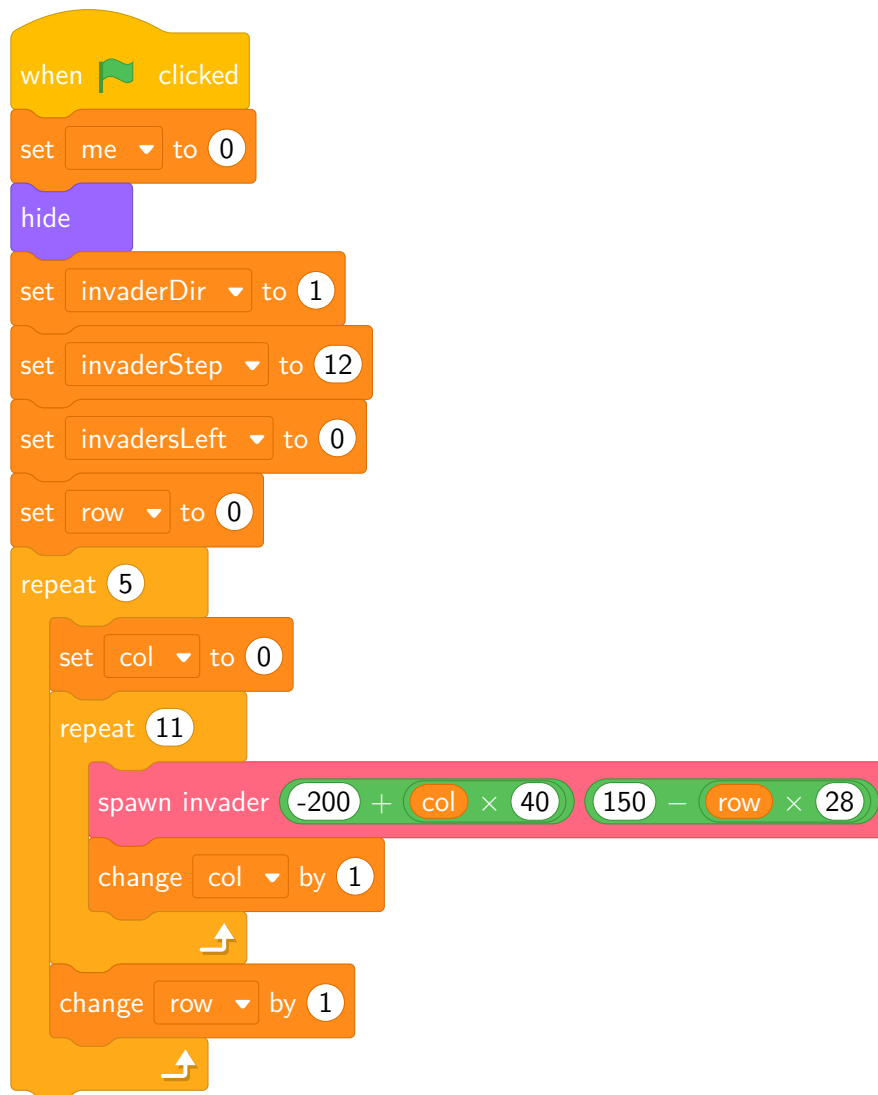
<pre>define spawn invader (x) (y)</pre>	Make it under <i>My Blocks -> Make a Block</i>; add two number inputs named x and y. The crimson header is where the block's own steps go.
<pre>go to x: (x) y: (y)</pre>	Position the original at the target spot; the clone is born THERE, so no pile-up.

Build it.

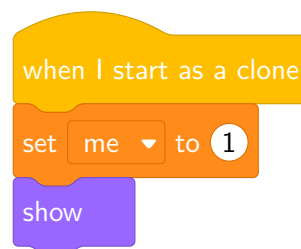
1. Make the block: *My Blocks -> Make a Block*, name it **spawn invader**, and add two number inputs **x** and **y**. Under the **define** header, add **go to x: (x) y: (y)**, **create clone of [myself]**, **change [invadersLeft] by (1)**.
2. On the same sprite, **when green flag clicked**: **set [me] to (0)** (this original is the conductor, not an invader), **hide**, and set **invaderDir** to (1), **invaderStep** to (12), **invadersLeft** to (0).
3. Build the grid with two **repeat** loops: **set [row] to (0)**, then **repeat (5) rows**; inside, **set [col] to (0)** then **repeat (11) columns** calling **spawn invader (x) (y)** with **x = (-200) + (col x 40)** and **y = (150) - (row x 28)**, then **change [col] by (1)**; after the inner loop **change [row] by (1)**.
4. Add the clone hat **when I start as a clone**: **set [me] to (1)** and **show**. Green flag - 55 invaders appear in formation.

The custom block (left), the grid builder (middle), and each clone waking up (right):





Conductor: build a 5x11 grid



Each clone: I am a real invader

A For this sprite only variable like *me* is copied into each clone and then owned by it - 55 separate copies. The hidden original keeps *me* = 0; every clone sets its own to 1. That one line is how the fleet's brain tells itself apart from its body.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Rank the rows by colour: in `spawn invader`, before cloning, `switch costume` by `row - row 0 invader_green`, row 4 `invader_yellow`, etc.

Expected: Each rank looks different, like the real game.

2. Change the formation: try `repeat (4)` rows of `repeat (8)` for a smaller, faster wave.

Expected: A different-sized fleet builds cleanly - the loops do the work.

Project 4: March as one

Make the fleet move as a single organism: a hidden conductor beats a `broadcast [march]`, every clone steps once, and if ANY clone touches the wall the conductor drops the fleet and reverses it. This is clone coordination - the heart of the project.

Get ready. No new sprites. Add the variable `hitEdge` For all sprites. You will extend the conductor's flag script and add two `when I receive` scripts to the Invader.

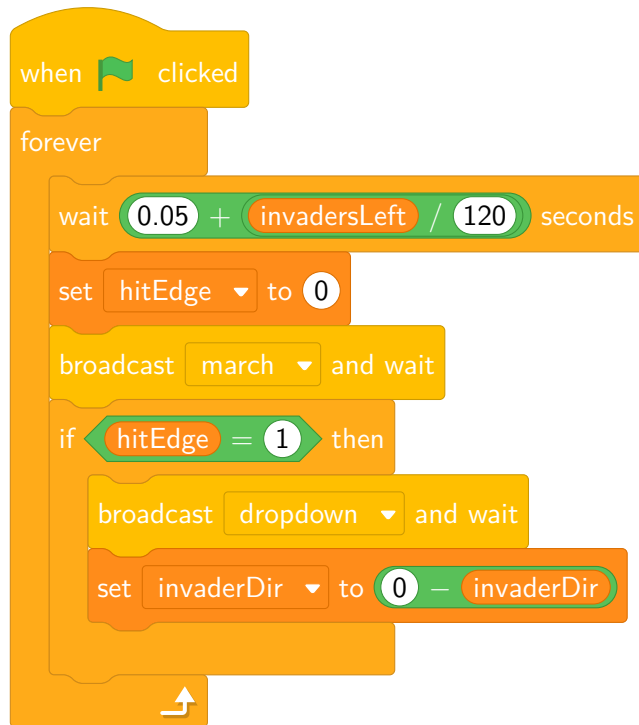
New blocks.

<code>broadcast [march] and wait</code>	Shout the beat and pause until every clone has finished its step - that is what keeps them in lockstep.
<code>(x position) x (invaderDir)</code>	One test that catches BOTH walls: going right it means $x > 215$; going left, $x < -215$.

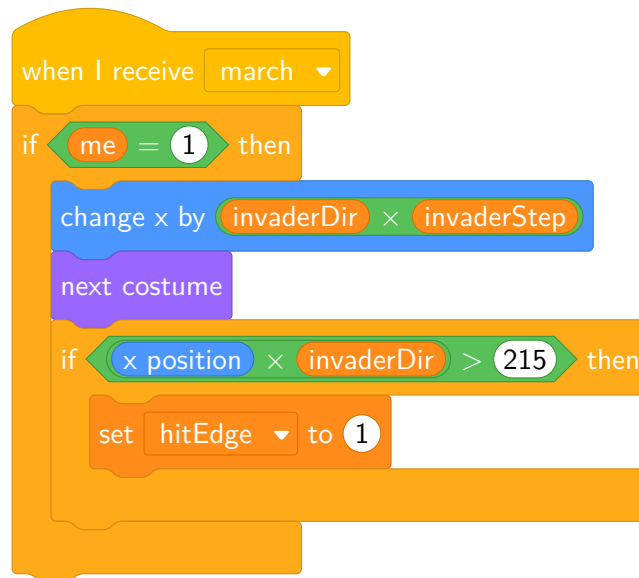
Build it.

1. At the bottom of the conductor's `when green flag clicked` script (under the grid loops), add a `forever`. Its blocks: `wait for (0.05) + (invadersLeft / 120)` seconds, then `set [hitEdge] to (0)` and `broadcast [march] and wait`.
2. Still in that loop: `if <(hitEdge) = (1)>` then `broadcast [dropdown] and wait` and `set [invaderDir] to ((0) - (invaderDir))`.
3. On the Invader, add `when I receive [march]`. Guard everything with `if <(me) = (1)>`. Inside: `change x by the amount (invaderDir) x (invaderStep)`, then `next costume`. Below those, `if` the edge test `(x position) x (invaderDir) > (215)` is true, `set [hitEdge] to (1)`.
4. Add `when I receive [dropdown]`, also guarded by `me`: `change y by (-16)`.
5. Green flag: the fleet marches, wiggles, and reverses at each wall.

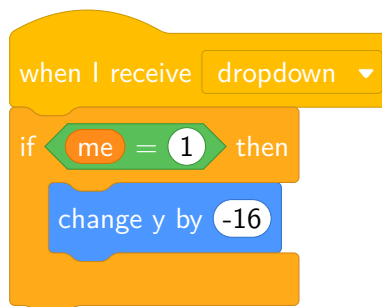
The conductor's heartbeat (left), and the two beats each invader obeys (right):



Conductor: beat the march (add under the grid loops)



Every invader: step, wiggle, watch the wall



Every invader: drop a row

Any ONE clone setting `hitEdge` is enough - the conductor reverses the WHOLE fleet once, on the next beat. If instead each clone flipped `invaderDir` itself, all 55 would fight and jitter. Coordination through a shared variable is the lesson.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Faster fleet: lower `invaderStep` to (8) for finer steps, or raise it to (16) for lurching jumps.

Expected: The march feel changes with one number.

2. Bigger drops: change `change y by (-16)` to (-24) so the fleet closes in faster.

Expected: Each wall bounce brings them down harder.

Project 5: Win, lose, and the famous speed-up

Finish the game: shoot invaders for points, win by clearing them, lose if they reach you or land - and feel the swarm accelerate as it thins. That speed-up is one line, and it is the most iconic bug in arcade history, copied on purpose.

Get ready. Add two text sprites: **GameOver** (`text_game_over`) and **YouWin** (`text_you_win`), both hidden at the start. Everything else is scripts on sprites you already have.

New blocks.

<code>touching [Bullet]?</code>	Each invader watches for a laser; when hit it scores, counts down, and removes itself.
<code>(invadersLeft) = (0)</code>	The win check - the same count that drives the speed-up also ends the game.

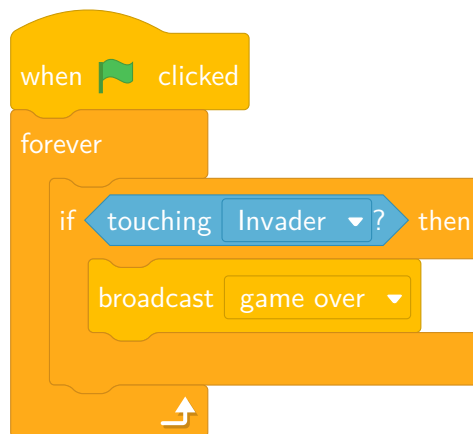
Build it.

1. On the Invader, add a second `when I start as a clone` with a `forever`: `if <touching [Bullet]> then change [score] by (10), change [invadersLeft] by (-1), start sound [invader die], delete this clone; and if <touching [Player]> then broadcast [game over]`.
2. In the `when I receive [dropdown]` script, after the drop add `if <(y position) < (-120)> then broadcast [game over]` - the fleet has landed.
3. In the conductor's heartbeat, at the bottom of the loop, add `if <(invadersLeft) = (0)> then broadcast [you win] and stop [all]`.
4. On the **Player**, add a `forever` that does `if <touching [Invader]> then broadcast [game over]`.
5. On **GameOver** and **YouWin**: `when green flag clicked hide`; and `when I receive [game over] / [you win] go to x: (0) y: (0), show, stop [all]`.
6. Green flag. Clear the fleet to win; let it touch you or land to lose - and notice the last few invaders are terrifyingly fast.

An invader taking a hit (left), the ship's loss check (middle), and the win screen (right):



Each invader: hit by a laser -> score and vanish



Player: touched by the fleet -> lose



YouWin: show the banner

The speed-up is the `wait ((0.05) + (invadersLeft / 120))` in the conductor: 55 invaders wait about half a second between beats; the last one waits a fifth of that. In 1978 this was an accident - the hardware drew fewer sprites faster - and it became the best difficulty curve in games. Here it is the same count that decides the win, reused for free.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Return fire: add a **Bomb** sprite. In an invader's loop, rarely (`if <(pick random (1) to (400)) = (1)>`) `go to` itself and `create clone of [Bomb]`; the bomb clone falls until it hits the Player.

Expected: Now the fleet shoots back.

2. Bonus mothership: add a **UFO** (`ufo_purple`) that slides across the top every 20 seconds for bonus points if you tag it.

Expected: A high-value target crosses the sky - pure Dino-style world-slide.

3. Rank scoring: give each invader a *this sprite only* `rank` set from `row` at spawn, and score `(10) + ((4) - (rank)) x (10)` so the back rows pay more.

Expected: The scariest back row is worth 50; the front row 10.
